

FOR JAVA DEVELOPERS

Recorded 3. Counted 6.

Object relationships in Java: inheritance, delegation, association, aggregation, composition and polymorphism, on one payment scenario

Java tells you which references and subtypes are legal. Your design says what they mean.

Sarah Thompson pays a shop. Each payment belongs to a customer, moves from created to authorised to captured, and keeps a history of those changes. At the end of the day the shop sends its payments to the bank in a settlement batch. Payments are written to a ledger, and each payment method charges its own fee. Amounts are whole US cents.

One package per relationship, each with a Demo you can run and tests that pin the behaviour. Where two designs are compared, both are in the code, and the versions that break are overlays the script applies to a scratch copy.

At the time of writing, October 2026, Java 25 is the current long term support release, and every run here used OpenJDK 25.0.4. The relationships are demonstrated on Java 25; none of them is new in it.

This sheet covers

- Inheritance, and why a subclass inherits how the parent's methods call each other
- Delegation through an interface, and exactly what it counts
- Association, shared aggregation and composition, told apart by three questions
- Polymorphism by overriding, against a sealed hierarchy with one switch

Simplified on purpose

- Amounts are whole cents in a long, not a money type with a currency
- The fees are made up and rounded down by integer division
- No null checks or other validation
- The status history lives in memory, not in a database

Code paths below are shortened to the part after `src/main/java/com/example/payments/`. Every block is a whole file at commit `0571e4f`, and every output is copied from `evidence/`.

01 The notation, on one table

Association: Payment knows its Customer; CountingLog knows a PaymentLog	Solid line, open arrowhead towards the object that is known
Shared aggregation: SettlementBatch groups Payments	Hollow diamond at the whole (the batch). UML leaves its exact meaning to your model
Composition: Payment owns its PaymentHistory	Filled diamond at the whole (the payment)
Inheritance: CountingLedger extends Ledger	Solid line, hollow triangle at the parent
Implementing an interface: Card implements PaymentMethod	Dashed line, hollow triangle at the interface
Polymorphism: Checkout.total calls fee()	No arrow of its own. The implements lines make it possible

Java has no keyword for association, aggregation or composition: all three compile to a field holding a reference. The diagram states which one you meant, and the code has to keep that promise.

02 Recorded 3. Counted 6.

Ledger.recordAll loops and calls record for each payment. CountingLedger overrides both methods to count. recordAll adds three, then super.recordAll runs the loop, and each record call made on this lands in the override, because this is a CountingLedger. Three more. Six.

A subclass inherits more than methods. It inherits the way the parent's methods call each other.

inheritance/PaymentLog.java

```
package com.example.payments.inheritance;

import java.util.List;

public interface PaymentLog {

    void record(Payment payment);

    void recordAll(List<Payment> payments);

    int size();
}
```

inheritance/Ledger.java

```
package com.example.payments.inheritance;

import java.util.ArrayList;
import java.util.List;

public class Ledger implements PaymentLog {

    private final List<Payment> entries = new ArrayList<>();

    @Override
    public void record(Payment payment) {
        entries.add(payment);
    }

    @Override
    public void recordAll(List<Payment> payments) {
        for (Payment payment : payments) {
            record(payment);
        }
    }

    @Override
    public int size() {
        return entries.size();
    }
}
```

```
inheritance/CountingLedger.java
```

```
package com.example.payments.inheritance;

import java.util.List;

// Inheritance: a Ledger that also counts what it records.
public class CountingLedger extends Ledger {

    private int recorded;

    @Override
    public void record(Payment payment) {
        recorded++;
        super.record(payment);
    }

    @Override
    public void recordAll(List<Payment> payments) {
        recorded += payments.size();
        super.recordAll(payments);
    }

    public int recorded() {
        return recorded;
    }
}
```

```
Three payments, recorded with recordAll (evidence/inheritance.log)
```

```
$ java com.example.payments.inheritance.Demo
CountingLedger: size 3, recorded 6
CountingLog:    size 3, recorded 3
```

Three payments through recordAll: the ledger holds 3, CountingLedger says 6.

Measured on a real run, 2026-10-08: evidence/ in the repository, reproduced by scripts/verify.sh.

03 The obvious fix, and the line that breaks it

Fix: count only in record, because recordAll calls record anyway. It gives 3 and the tests pass. Then the owner of Ledger replaces the loop with one addAll, a reasonable change that nothing in Ledger's contract forbids. The same subclass now counts 0, and not one line of it changed.

The six was a mistake in the subclass. The zero is the deeper problem: a sensible fix that silently relied on the parent's internals.

```
overlays/count-in-record-only: inheritance/CountingLedger.java
```

```
package com.example.payments.inheritance;

// The obvious fix: only count in record(), because
// recordAll() already calls record() for every payment.
public class CountingLedger extends Ledger {

    private int recorded;

    @Override
    public void record(Payment payment) {
        recorded++;
        super.record(payment);
    }

    public int recorded() {
        return recorded;
    }
}
```

```
overlays/ledger-uses-addall: inheritance/Ledger.java
```

```
package com.example.payments.inheritance;

import java.util.ArrayList;
import java.util.List;

public class Ledger implements PaymentLog {

    private final List<Payment> entries = new ArrayList<>();

    @Override
    public void record(Payment payment) {
        entries.add(payment);
    }

    @Override
    public void recordAll(List<Payment> payments) {
        entries.addAll(payments);
    }

    @Override
    public int size() {
        return entries.size();
    }
}
```

```
The fix alone (evidence/inheritance-record-only.log)
```

```
# overlay: count-in-record-only
$ ./mvnw compile
BUILD SUCCESS
$ java com.example.payments.inheritance.Demo
CountingLedger: size 3, recorded 3
CountingLog:    size 3, recorded 3
```

```
The fix, after the parent changed (evidence/inheritance-base-changed.log)
```

```
# overlay: count-in-record-only
# overlay: ledger-uses-addall
$ ./mvnw compile
BUILD SUCCESS
$ java com.example.payments.inheritance.Demo
CountingLedger: size 3, recorded 0
CountingLog:    size 3, recorded 3
```

CountingLedger: 6, then 3 with the fix, then 0 once Ledger.recordAll uses addAll. CountingLog: 3 in every case.

Measured on a real run, 2026-10-08: evidence/ in the repository, reproduced by scripts/verify.sh.



04 Delegation: the same goal without extending anything

CountingLog implements PaymentLog and holds a PaymentLog. Every call is forwarded, and the count goes up only after the log has accepted the call. It depends on what the interface promises, not on how a Ledger implements it: 3 against the original ledger, 3 against the one that switched to addAll.

What it counts is successful record and recordAll calls. If the log refuses a payment by throwing, nothing is counted, and a test pins that. If other code writes to the same ledger directly, this counter does not see it.

This is what "composition over inheritance" usually means: building behaviour out of objects. On a class diagram, though, CountingLog to PaymentLog is an association with delegation, not UML composition. Whoever created the ledger can still hold it. And the constructor parameter does not decide the relationship: injection is how an object arrives, not who else holds it.

```
inheritance/CountingLog.java

package com.example.payments.inheritance;

import java.util.List;

// Delegation: holds a PaymentLog and forwards to it,
// counting a call only once the log has accepted it.
public final class CountingLog implements PaymentLog {

    private final PaymentLog log;
    private int recorded;

    public CountingLog(PaymentLog log) {
        this.log = log;
    }

    @Override
    public void record(Payment payment) {
        log.record(payment);
        recorded++;
    }

    @Override
    public void recordAll(List<Payment> payments) {
        log.recordAll(payments);
        recorded += payments.size();
    }

    @Override
    public int size() {
        return log.size();
    }

    public int recorded() {
        return recorded;
    }
}
```

CountingLog says 3 against the original Ledger and 3 against the Ledger that uses addAll.

Measured on a real run, 2026-10-08: evidence/ in the repository, reproduced by scripts/verify.sh.

A log that throws on every write leaves CountingLog.recorded() at zero
(CountingTest.theWrapperDoesNotCountAPaymentTheLogRefused).

Pinned by the repository tests, 19 of 19 passing in evidence/tests.log, 2026-10-08.

05 Inheritance used well: exception specialisation

A payment state exception is an illegal state exception, and every catch written for one still catches it. That is the test for inheritance: a real "is a", where the subclass can stand in for the parent, and a parent that was designed to be extended.

composition/PaymentStateException.java

```
package com.example.payments.composition;

// Inheritance that works: a payment state error IS an
// IllegalStateException, and every catch written for one
// still catches it.
public class PaymentStateException
    extends IllegalStateException {

    public PaymentStateException(String message) {
        super(message);
    }
}
```

Capturing a payment that was never authorised (evidence/exception.log)

```
$ java com.example.payments.composition.ExceptionDemo
caught by: catch (IllegalStateException e)
class:     PaymentStateException
message:   P-1 is CREATED, not AUTHORISED
```

capture() on a CREATED payment throws PaymentStateException, caught by catch (IllegalStateException e).

Measured on a real run, 2026-10-08: evidence/ in the repository, reproduced by scripts/verify.sh.

06 Three questions, not a keyword

1. Is this a whole and its parts, or two objects that only know each other?
2. Can one part belong to more than one whole at the same time?
3. When the whole is deleted, does the part go with it?

How the object arrived, through a constructor or anywhere else, is a clue. It is not the answer. And logical ownership is not object lifetime: Java has no destructors, the garbage collector can reclaim an object only once nothing can reach it, and it decides when. A class diagram does not predict that, and does not need to.

Association, shared aggregation and composition are distinguished by what the design promises, not by Java syntax; all three are a field holding a reference.

A design judgement, stated as such, not a measurement.

07 Association: a payment knows its customer

Two objects that know each other, and nothing more. A customer is not made of payments, and a payment is not part of a customer. Customer has no list of payments, so the arrow points one way: one customer, zero or more payments.

```
association/Customer.java
```

```
package com.example.payments.association;

public record Customer(String id, String name) {}
```

```
association/Payment.java
```

```
package com.example.payments.association;

// A payment knows its customer. The customer was created
// first, by someone else, and does not know its payments.
public record Payment(String id, Customer customer,
    long cents) {}
```

```
Two payments for Sarah (evidence/association.log)
```

```
$ java com.example.payments.association.Demo
P-1 customer: Sarah Thompson
P-2 customer: Sarah Thompson
same Customer object: true
```

P-1 and P-2 hold the same Customer object.

Measured on a real run, 2026-10-08: evidence/ in the repository, reproduced by scripts/verify.sh.



08 Aggregation: a batch groups payments that outlive it

The batch is a whole and the payments are its parts, but a part can belong to more than one batch over time, and rejecting the batch does not delete its payments. Shared aggregation: a hollow diamond at the batch end.

UML 2.5.1 deliberately leaves the precise meaning of shared aggregation to the application area and the modeler. It is a modelling choice you make and write down, not something Java works out from a `List<Payment>`. A real system would also check that no payment is ever settled twice.

aggregation/SettlementBatch.java

```
package com.example.payments.aggregation;

import java.util.ArrayList;
import java.util.List;

// Groups payments to send to the bank at the end of the day.
// The payments existed before the batch and outlive it.
public final class SettlementBatch {

    private final String id;
    private final List<Payment> payments = new ArrayList<>();
    private boolean rejected;

    public SettlementBatch(String id) {
        this.id = id;
    }

    public void add(Payment payment) {
        payments.add(payment);
    }

    public void reject() {
        rejected = true;
    }

    public boolean rejected() {
        return rejected;
    }

    public long totalCents() {
        return payments.stream()
            .mapToLong(Payment::cents)
            .sum();
    }

    public List<Payment> payments() {
        return List.copyOf(payments);
    }

    public String id() {
        return id;
    }
}
```



```
Monday rejected, then a retry (evidence/aggregation.log)
```

```
$ java com.example.payments.aggregation.Demo
MON: 2 payments, 6500 cents
MON rejected: true
P-1 unchanged: P-1, 2500 cents
MON-RETRY: 2 payments, 6500 cents
same P-1 object in both batches: true
```

MON: 2 payments, 6500 cents, rejected. P-1 unchanged at 2500 cents. MON-RETRY holds the same P-1 object.

Measured on a real run, 2026-10-08: evidence/ in the repository, reproduced by scripts/verify.sh.

The precise semantics of shared aggregation are left to the application area and the modeler.

OMG Unified Modeling Language 2.5.1, AggregationKind: shared aggregation's precise semantics vary by application area and modeler.



09 Composition: same fields, a different owner

Payment creates its PaymentHistory as a field, changes it only through `authorise()` and `capture()`, and hands out only a copy. A filled diamond at the payment end: a history belongs to exactly one payment, and goes with it.

SharedHistoryPayment is the same class, except the history comes in through the constructor. Build two with one history, authorise the first, and the second says `AUTHORISED`. It never was. Java allowed it; the diagram did not. The code has to enforce the ownership the diagram claims.

composition/Payment.java

```
package com.example.payments.composition;

import java.util.List;

// Owns its history: it creates it, changes it only through
// authorise() and capture(), and never hands it out.
public final class Payment {

    private final String id;
    private final Customer customer;
    private final long cents;
    private final PaymentHistory history =
        new PaymentHistory();

    public Payment(String id, Customer customer,
        long cents) {
        this.id = id;
        this.customer = customer;
        this.cents = cents;
    }

    public void authorise() {
        move(Status.CREATED, Status.AUTHORISED);
    }

    public void capture() {
        move(Status.AUTHORISED, Status.CAPTURED);
    }

    public Status status() {
        return history.current();
    }

    public List<StatusChange> history() {
        return history.entries();
    }

    private void move(Status from, Status to) {
        if (status() != from) {
            throw new PaymentStateException(
                id + " is " + status() + ", not " + from);
        }
        history.add(new StatusChange(from, to));
    }
}
```

composition/SharedHistoryPayment.java

```
package com.example.payments.composition;

import java.util.List;

// Takes its history from the caller, so two payments can
// be given the same one.
public final class SharedHistoryPayment {

    private final String id;
    private final Customer customer;
    private final long cents;
    private final PaymentHistory history;

    public SharedHistoryPayment(String id, Customer customer,
                                long cents,
                                PaymentHistory history) {
        this.id = id;
        this.customer = customer;
        this.cents = cents;
        this.history = history;
    }

    public void authorise() {
        move(Status.CREATED, Status.AUTHORISED);
    }

    public void capture() {
        move(Status.AUTHORISED, Status.CAPTURED);
    }

    public Status status() {
        return history.current();
    }

    public List<StatusChange> history() {
        return history.entries();
    }

    private void move(Status from, Status to) {
        if (status() != from) {
            throw new PaymentStateException(
                id + " is " + status() + ", not " + from);
        }
        history.add(new StatusChange(from, to));
    }
}
```

```
composition/PaymentHistory.java
```

```
package com.example.payments.composition;

import java.util.ArrayList;
import java.util.List;

// The status changes of one payment, in order.
public final class PaymentHistory {

    private final List<StatusChange> changes =
        new ArrayList<>();

    void add(StatusChange change) {
        changes.add(change);
    }

    public Status current() {
        return changes.isEmpty()
            ? Status.CREATED
            : changes.getLast().to();
    }

    public List<StatusChange> entries() {
        return List.copyOf(changes);
    }
}
```

Status is an enum of CREATED, AUTHORISED and CAPTURED, and StatusChange is a record of from and to. add is package private, and entries returns a copy.

```
P-1 and P-2 share one history; P-3 and P-4 own theirs (evidence/composition.log)
```

```
$ java com.example.payments.composition.Demo
SharedHistoryPayment P-1: AUTHORISED
SharedHistoryPayment P-2: AUTHORISED
Payment P-3: AUTHORISED
Payment P-4: CREATED
```

Shared history: authorising P-1 makes P-2 report AUTHORISED. Owned history: P-3 AUTHORISED, P-4 still CREATED.

Measured on a real run, 2026-10-08: evidence/ in the repository, reproduced by scripts/verify.sh.

Adding to the list returned by history() throws UnsupportedOperationException and leaves the payment CREATED (PaymentTest.theHistoryItHandsOutCannotBeChanged).

Pinned by the repository tests, 19 of 19 passing in evidence/tests.log, 2026-10-08.

10 This fee was 0 cents and nothing failed

One method decides every fee from a string. Card is 20 cents plus one and a half percent, bank transfer a flat 30 cents, and anything else falls through to zero: mobile money, and a typo with the e and the n swapped. It compiles and nothing fails. An enum, or throwing on an unknown name, would also catch this. Polymorphism fixes it a different way.

```
polymorphism/strings/FeeCalculator.java
```

```
package com.example.payments.polymorphism.strings;

// One method decides the fee for every kind of payment.
public class FeeCalculator {

    public long fee(String method, long cents) {
        if (method.equals("CARD")) {
            return 20 + cents * 15 / 1000;
        } else if (method.equals("BANK_TRANSFER")) {
            return 30;
        }
        return 0;
    }
}
```

```
evidence/fees-strings.log
```

```
$ java com.example.payments.polymorphism.strings.Demo
CARD fee on 2500 cents: 57 cents
BANK_TRANSFER fee on 2500 cents: 30 cents
MOBILE_MONEY fee on 2500 cents: 0 cents
MOBILE_MOENY fee on 2500 cents: 0 cents
```

MOBILE_MONEY and MOBILE_MOENY both cost 0 cents on 2500, with no error.

Measured on a real run, 2026-10-08: evidence/ in the repository, reproduced by scripts/verify.sh.

11 Overriding: one call, the object decides

PaymentMethod is an interface with one method, fee. Each kind of payment implements its own rule, and Checkout.total calls fee without ever asking which one it has. On the diagram polymorphism has no arrow of its own: the dashed implements lines show the types that make it possible.

Add a payment method and forget its fee, and it does not compile. That is the interface contract: every concrete class that implements PaymentMethod must provide fee. This PaymentMethod is not sealed, so the error has nothing to do with sealing.

```
polymorphism/overriding/PaymentMethod.java
```

```
package com.example.payments.polymorphism.overriding;

// Every kind of payment says what it costs.
public interface PaymentMethod {

    long fee(long cents);
}
```

```
polymorphism/overriding/Checkout.java
```

```
package com.example.payments.polymorphism.overriding;

// Never asks which kind of payment it has.
public class Checkout {

    public long total(PaymentMethod method, long cents) {
        return cents + method.fee(cents);
    }
}
```

```
polymorphism/overriding/Card.java
```

```
package com.example.payments.polymorphism.overriding;

public record Card(String last4) implements PaymentMethod {

    @Override
    public long fee(long cents) {
        return 20 + cents * 15 / 1000;
    }
}
```

```
evidence/fees-overriding.log
```

```
$ java com.example.payments.polymorphism.overriding.Demo
Card total on 2500 cents: 2557 cents
BankTransfer total on 2500 cents: 2530 cents
MobileMoney total on 2500 cents: 2550 cents
```

MobileMoney.fee returns cents * 2 / 100 and BankTransfer.fee returns 30; both are in the repository beside Card.

```
overlays/mobile-money-without-fee: polymorphism/overriding/MobileMoney.java
```

```
package com.example.payments.polymorphism.overriding;

public record MobileMoney(String phone)
    implements PaymentMethod {}
```

```
evidence/overriding-missing-fee.log
```

```
# overlay: mobile-money-without-fee
$ ./mvnw compile
src/main/java/com/example/payments/polymorphism/overriding/MobileMoney.java:[3,8]
com.example.payments.polymorphism.overriding.MobileMoney is not abstract and does not override
abstract method fee(long) in com.example.payments.polymorphism.overriding.PaymentMethod
BUILD FAILURE
```

Totals on 2500 cents: Card 2557, BankTransfer 2530, MobileMoney 2550. A MobileMoney with no fee() fails to compile: it "is not abstract and does not override abstract method fee(long)".

Measured on a real run, 2026-10-08: evidence/ in the repository, reproduced by scripts/verify.sh.

12 The alternative: a sealed hierarchy and one switch

The payment methods are only data: a sealed interface permitting exactly three records, and every fee rule in one switch with no default branch. Leave out mobile money and the compiler refuses. That is a second, different protection: a sealed hierarchy plus a switch expression that must cover every case.

Which one? Overriding usually keeps the behaviour for one type together; a switch usually keeps one operation together. Both have extension costs: with overriding a new rule touches every class, with the switch a new kind of payment touches every switch. Ask what changes more often in your code, new kinds of payment or new rules about them.

```
polymorphism/switching/PaymentMethod.java
```

```
package com.example.payments.polymorphism.switching;

// The kinds of payment are data. The fee lives elsewhere.
public sealed interface PaymentMethod
    permits Card, BankTransfer, MobileMoney {}

Card, BankTransfer and MobileMoney are records here with no methods at all.
```

```
polymorphism/switching/Fees.java
```

```
package com.example.payments.polymorphism.switching;

// Every fee rule in one place, checked by the compiler.
public final class Fees {

    private Fees() {}

    public static long fee(PaymentMethod method, long cents) {
        return switch (method) {
            case Card card -> 20 + cents * 15 / 1000;
            case BankTransfer transfer -> 30;
            case MobileMoney mobile -> cents * 2 / 100;
        };
    }
}
```

```
evidence/fees-switching.log
```

```
$ java com.example.payments.polymorphism.switching.Demo
Card fee on 2500 cents: 57 cents
BankTransfer fee on 2500 cents: 30 cents
MobileMoney fee on 2500 cents: 50 cents
```

```
overlays/switch-without-mobile-money removes the MobileMoney case (evidence/switch-missing-case.log)
```

```
# overlay: switch-without-mobile-money
$ ./mvnw compile
src/main/java/com/example/payments/polymorphism/switching/Fees.java:[9,16] the switch
expression does not cover all possible input values
BUILD FAILURE
```

Fees on 2500 cents: Card 57, BankTransfer 30, MobileMoney 50, the same as the overrides. Without the MobileMoney case: "the switch expression does not cover all possible input values".

Measured on a real run, 2026-10-08: evidence/ in the repository, reproduced by scripts/verify.sh.

Overriding against a switch is a trade-off between which change is cheap, not a rule that one is better.

A design judgement, stated as such, not a measurement.

Naming a relationship in a design review

1. Is this a whole and its parts, or two objects that only know each other? If they only know each other, it is association
2. Can one part belong to more than one whole at the same time, or move between wholes? Then it is not composition
3. When the whole is deleted, does the part go with it? Only then draw the filled diamond, and make the code create the part and keep it
4. Do not let constructor injection decide: it is how an object arrives, not who else holds it
5. Do not read a diagram as a memory model: the garbage collector reclaims what nothing can reach, when it chooses
6. Extend a class only for a real "is a" that can stand in for the parent, from a parent designed to be extended
7. To add behaviour to something you do not own, implement its interface and forward to it, and say exactly what your wrapper sees
8. Choose overriding when new kinds change most often, a switch over a sealed hierarchy when new rules do



Everything else

-  **Site**
<https://code-with-sam-dev.github.io>
-  **YouTube**
<https://www.youtube.com/@CodewithSam-Dev>
-  **LinkedIn**
<https://www.linkedin.com/company/code-with-sam-dev>
-  **x**
<https://x.com/CodeWithSamDev>
-  **TikTok**
<https://www.tiktok.com/@codewithsamdev>
-  **Instagram**
<https://www.instagram.com/codewithsamdev>
-  **Facebook**
<https://www.facebook.com/codewithsamdev>
-  **GitHub**
<https://github.com/code-with-sam-dev>
-  **Buy me a coffee**
<https://buymeacoffee.com/codewithsam>
-  **Buy me sadza**
<https://buymesadza.com/codewithsam>
- The repository: every package, the overlays, the tests and scripts/verify.sh**
<https://github.com/code-with-sam-dev/java-object-relationships>
- OMG: Unified Modeling Language 2.5.1**
<https://www.omg.org/spec/UML/2.5.1>
- The Java Language Specification, Java SE 25**
<https://docs.oracle.com/javase/specs/jls/se25/html/index.html>
- JEP 409: Sealed Classes**
<https://openjdk.org/jeps/409>
- JEP 441: Pattern Matching for switch**
<https://openjdk.org/jeps/441>
- JEP 395: Records**
<https://openjdk.org/jeps/395>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which design topic you want taken apart next.

Java is a trademark of Oracle. UML and Unified Modeling Language are trademarks of the Object Management Group. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by either of them.

Technical claims verified 2026-10-08 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.